

PIPELINING

UNIT-V

PIPELINING

- Basic concepts of pipelining
- Efficiency, throughput, and speedup
- Pipeline hazards and types
- Approaches to overcome pipeline hazards
- Parallel Processors: Introduction to parallel processors, FLYNN's classification, Cache Coherence problem,
- Multiprocessors
- Multi-Core system
- Introduction to Graphics Processing Unit

Basic concepts of pipelining

- Pipelining is a computer architecture technique that improves a processor's performance by breaking down instruction execution into smaller, parallel stages, similar to an assembly line.
- Instead of completing one instruction before starting the next, multiple instructions are processed simultaneously, with each instruction moving through its own stage at the same time, which increases overall instruction throughput.

Benefits

- **Higher performance:** Pipelining significantly increases the processor's instruction throughput, meaning more instructions are completed in a given amount of time.
- **Efficient hardware use:** It allows different parts of the processor to work in parallel, maximizing the utilization of the hardware.

Basic concepts of pipelining

- Pipelining is a technique for breaking down a sequential process into various sub-operations and executing each sub-operation in its own dedicated segment that runs in parallel with all other segments.
- The most significant feature of a pipeline technique is that it allows several computations to run in parallel in different parts at the same time.
- By associating a register with every segment in the pipeline, the process of computation can be made overlapping.
- The registers provide separation and isolation among every segment, allowing each to work on different data at the same time.
- An input register for each segment, followed by a combinational circuit, can be used to illustrate the structure of a pipeline organisation.
- To better understand the pipeline organisation, consider an example of a combined multiplication and addition operation.

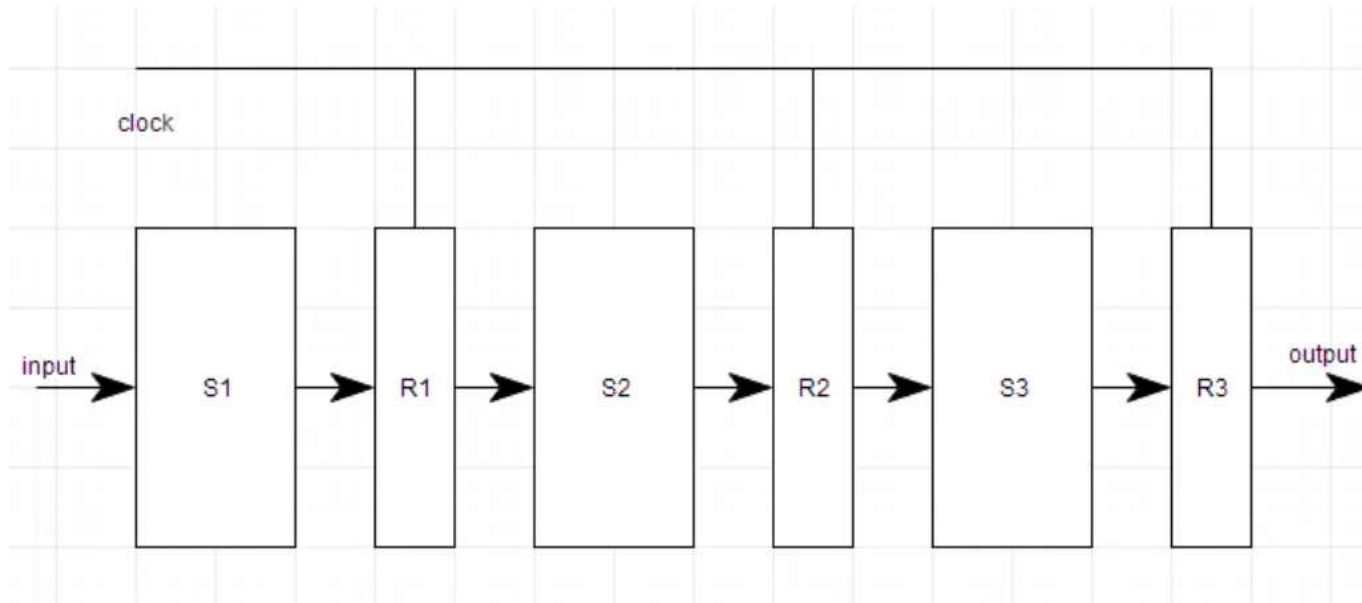
Basic concepts of pipelining

Design of a basic Pipeline

- In a pipelined processor, a pipeline has two ends, the input end and the output end. Between these ends, there are multiple stages/segments such that the output of one stage is connected to the input of the next stage and each stage performs a specific operation.
- Interface registers are used to hold the intermediate output between two stages.
- These interface registers are also called latch or buffer.
- All the stages in the pipeline along with the interface registers are controlled by a common clock.
- Execution in a pipelined processor Execution sequence of instructions in a pipelined processor can be visualized using a space-time diagram.
- For example, consider a processor having 4 stages and let there be 2 instructions to be executed.

Basic concepts of pipelining

- In pipeline system, each segment consists of an input register followed by a combinational circuit.
- The register is used to hold data and combinational circuit performs operations on it.
- The output of combinational circuit is applied to the input register of the next segment.



Basic concepts of pipelining

Types of Pipeline

- It is divided into 2 categories:

1. Arithmetic Pipeline
2. Instruction Pipeline

1. Arithmetic Pipeline

Arithmetic pipelines are usually found in most of the computers. They are used for floating point operations, multiplication of fixed point numbers etc. For example: The input to the Floating Point Adder pipeline is.

$$X = A * 2^a$$

$$Y = B * 2^b$$

Here A and B are mantissas (significant digit of floating point numbers), while a and b are exponents.

The floating point addition and subtraction is done in 4 parts:

Compare the exponents, Align the mantissas, Add or subtract mantissas, Produce the result.

Basic concepts of pipelining

Instruction Pipeline

- In this a stream of instructions can be executed by overlapping fetch, decode and execute phases of an instruction cycle.
- This type of technique is used to increase the throughput of the computer system.
- An instruction pipeline reads instruction from the memory while previous instructions are being executed in other segments of the pipeline.
- Thus we can execute multiple instructions simultaneously.
- The pipeline will be more efficient if the instruction cycle is divided into segments of equal duration.

Basic concepts of pipelining

Pipeline Conflicts

- There are some factors that cause the pipeline to deviate its normal performance. Some of these factors are given below:

1. Timing Variations

- All stages cannot take same amount of time. This problem generally occurs in instruction processing where different instructions have different operand requirements and thus different processing time.

2. Data Hazards

- When several instructions are in partial execution, and if they reference same data then the problem arises. We must ensure that next instruction does not attempt to access data before the current instruction, because this will lead to incorrect results.

3. Branching

- In order to fetch and execute the next instruction, we must know what that instruction is. If the present instruction is a conditional branch, and its result will lead us to the next instruction, then the next instruction may not be known until the current one is processed.

Basic concepts of pipelining

4. Interrupts

- Interrupts set unwanted instruction into the instruction stream. Interrupts effect the execution of instruction.

5. Data Dependency

- It arises when an instruction depends upon the result of a previous instruction but this result is not yet available.

Advantages of Pipelining

- The cycle time of the processor is reduced.
- It increases the throughput of the system
- It makes the system reliable.

Disadvantages of Pipelining

- The design of pipelined processor is complex and costly to manufacture.
- The instruction latency is more.

Efficiency, throughput, and speedup

- Efficiency, throughput, and speedup are performance metrics used to evaluate how well a system works.
- Throughput measures the rate of work completed per unit of time, speedup quantifies how much faster a system is compared to a baseline, and efficiency compares the actual speedup to the ideal speedup, indicating how effectively resources are being used.
- **Efficiency**
- **What it is:** The ratio of how fast a system is working to the ideal maximum speedup. It shows how well resources are being utilized.

Formula:

- $\text{Efficiency} = \text{Speedup} / \text{Number of stages}$
- $\text{Efficiency} = (\text{Actual execution time} / \text{Ideal execution time})$
- **Example:** In a pipelined processor, efficiency measures how much of the pipeline's potential is being realized, considering potential delays and bottlenecks.

Efficiency, throughput, and speedup

Throughput

- **What it is:** The rate at which a system completes tasks or processes data over a given period. It's a measure of the system's output capacity.

Formula:

- **Throughput** = Number of instructions / Total time to complete instructions
- **Example:** A network router's throughput is the total amount of data it can transmit per second.

Speedup

- **What it is:** A measure of how much faster a parallel or pipelined system is compared to a single-processor or non-pipelined system.
- **Formula: Speedup** = Sequential execution time / Parallel execution time.
- **Example:** If a task takes 100 seconds on a single processor and only 20 seconds on a four-processor system, the speedup is $(100/20=5)$.

Pipeline hazards and types

- As we all know, the CPU's speed is limited by memory.
- There's one more case to consider, i.e. a few instructions are at some stage of execution in a pipelined design.
- There is a chance that these sets of instructions will become dependent on one another, reducing the pipeline's pace.
- Dependencies arise for a variety of reasons, which we will examine shortly.
- The dependencies in the pipeline are referred to as hazards since they put the execution at risk.
- We can swap the terms, dependencies and hazards since they are used interchangeably in computer architecture.
- A hazard, in essence, prevents an instruction present in the pipe from being performed during the specified clock cycle.
- Since each of the instructions may be in a separate machine cycle, we use the term clock cycle.

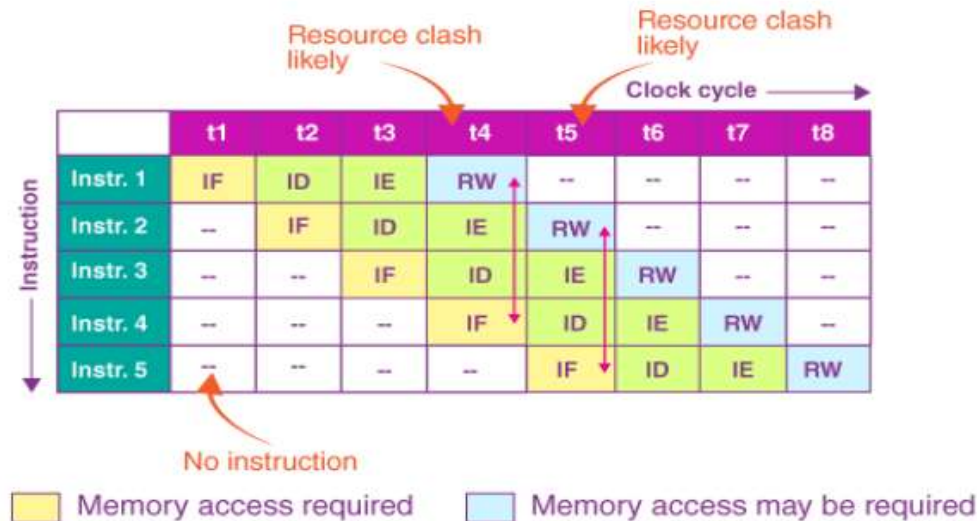
Types of Pipeline Hazards in Computer Architecture

- The three different types of hazards in computer architecture are:
 1. Structural
 2. Data
 3. Control
- Dependencies can be addressed in a variety of ways.
- The easiest is to introduce a bubble into the pipeline, which stalls it and limits throughput.
- The bubble forces the next instruction to wait until the previous one is completed.

Types of Pipeline Hazards in Computer Architecture

Structural Hazard

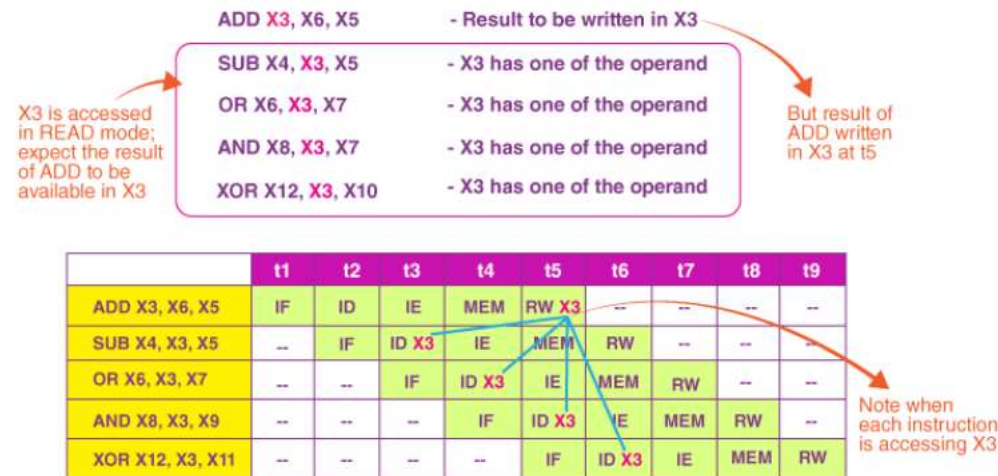
- Hardware resource conflicts among the instructions in the pipeline cause structural hazards. Memory, a GPR Register, or an ALU might all be used as resources here.
- When more than one instruction in the pipe requires access to the very same resource in the same clock cycle, a resource conflict is said to arise.
- In an overlapping pipelined execution, this is a circumstance where the hardware cannot handle all potential combinations.



Types of Pipeline Hazards in Computer Architecture

Data Hazards

- Data hazards in pipelining emerge when the execution of one instruction is dependent on the results of another instruction that is still being processed in the pipeline.
- The order of the READ or WRITE operations on the register is used to classify data threats into three groups.



Types of Pipeline Hazards in Computer Architecture

Control Hazards

- Branch hazards are caused by branch instructions and are known as control hazards in computer architecture.
- The flow of program/instruction execution is controlled by branch instructions.
- Remember that conditional statements are used in higher-level languages for iterative loops and condition testing (correlate with while, for, and if case statements).
- These are converted into one of the BRANCH instruction variations.
- As a result, when the decision to execute one instruction is reliant on the result of another instruction, such as a conditional branch, which examines the condition's consequent value, a conditional hazard develops.



Approaches to overcome pipeline hazards

Control Hazards

- Branch hazards are caused by branch instructions and are known as control hazards in computer architecture.
- The flow of program/instruction execution is controlled by branch instructions.
- Remember that conditional statements are used in higher-level languages for iterative loops and condition testing (correlate with while, for, and if case statements).
- These are converted into one of the BRANCH instruction variations.
- As a result, when the decision to execute one instruction is reliant on the result of another instruction, such as a conditional branch, which examines the condition's consequent value, a conditional hazard develops.



Parallel Processors: Introduction to parallel processors

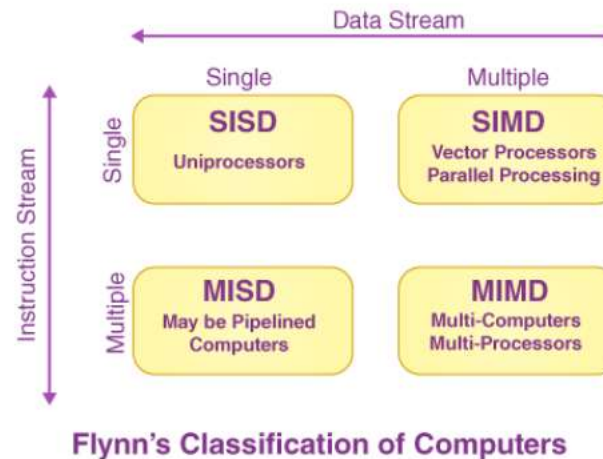
- Parallel processing is used to increase the computational speed of computer systems by performing multiple data-processing operations simultaneously.
- For example, while an instruction is being executed in ALU, the next instruction can be read from memory.
- The system can have two or more ALUs and be able to execute multiple instructions at the same time.
- In addition, two or more processing is also used to speed up computer processing capacity and increases with parallel processing, and with it, the cost of the system increases.
- But, technological development has reduced hardware costs to the point where parallel processing methods are economically possible.
- Parallel processing derives from multiple levels of complexity. It is distinguished between parallel and serial operations by the type of registers used at the lowest level.

Introduction to parallel processors

- Parallel processors are systems with multiple processing units that execute instructions simultaneously to perform tasks concurrently, significantly improving speed and efficiency over single-processor systems.
- This is achieved by dividing a large problem into smaller subtasks that are processed at the same time, using techniques like data parallelism and task parallelism.
- Parallel processing is essential for handling complex computations in areas like AI, machine learning, and high-performance computing.
- **Types of parallelism**
- Task parallelism: Different tasks or processes are executed simultaneously on different processors.
- Data parallelism: The same operation is performed on different parts of a dataset at the same time.
- Instruction-level parallelism: Multiple instructions from a single program are executed in parallel.
- Vectorization: A process where multiple, similar tasks are completed at once, often used in graphics processing units (GPUs).

FLYNN's classification

- Flynn's classification is a system for categorizing computer architectures based on the number of concurrent instruction and data streams they can handle. The four categories are SISD (Single Instruction, Single Data), SIMD (Single Instruction, Multiple Data), MISD (Multiple Instruction, Single Data), and MIMD (Multiple Instruction, Multiple Data).



FLYNN's classification

- **SISD (Single Instruction, Single Data):** This is the traditional, sequential processing architecture, like a standard personal computer with a single processor.
- **SIMD (Single Instruction, Multiple Data):** A single control unit executes the same instruction on multiple data streams simultaneously, used in applications like vector processing.
- **MISD (Multiple Instruction, Single Data):** This is a theoretical category where multiple instructions process a single data stream. While not practically implemented, some fault-tolerant systems are considered to be in this category, like redundant systems.
- **MIMD (Multiple Instruction, Multiple Data):** This is a parallel processing architecture where multiple processors can execute different instructions on different data streams at the same time, such as in most modern multiprocessor computers.

Cache Coherence problem

- Cache coherence issues arise in multi-processor systems when multiple caches hold a copy of the same memory location, and one cache modifies its copy, leading to an inconsistent view of the data across the system.
- This is resolved using cache coherence protocols that coordinate memory operations, ensuring all caches are updated or invalidated when shared data changes, common approaches being snoopy protocols and directory schemes.
- The problem: In a system with multiple processors, each can have its own local cache. If a piece of data from main memory is copied into multiple caches, and one processor modifies its local copy, the other caches will have an outdated, inconsistent version of that data.

Cache Coherence problem

- Cache coherence protocols ensure that all processors have a consistent view of shared data.
- **Snoopy protocols:** Processors monitor a shared bus for memory transactions. When a processor writes to a memory block, its cache "snoops" this write and either invalidates its own copy or requests an update from the writing processor, depending on the protocol's rules.
- A common snoopy protocol is MESI, which uses four states (Modified, Exclusive, Shared, and Invalid) to track the status of each cache line and maintain coherence.
- **Directory schemes:** Instead of a broadcast bus, a central directory keeps track of which caches hold a copy of each memory block. When a write occurs, the directory is consulted to find and update only the specific caches that need the new data, which is more scalable for systems with many processors.

Cache Coherence problem

- **Write invalidation:** A common strategy where a processor writing to a shared cache line broadcasts an invalidation message to all other caches that hold a copy of that line. Those caches then invalidate their copies so they must fetch the new version when they next need it.
- **False sharing:** A specific performance issue that can occur when multiple processors share parts of the same cache block but not the same bytes within that block. This can lead to constant invalidation and updates, a problem often called "ping-ponging," which can be mitigated by careful data placement.

Multiprocessors

- A multiprocessor is a computer system with two or more central processing units (CPUs) that work together to increase a computer's performance and speed. These CPUs can share a single main memory (shared memory) or each have their own private memory (distributed memory). This setup allows for the parallel execution of multiple tasks simultaneously, leading to faster processing, better multitasking, and increased reliability.
- **Key characteristics**
- **Parallel processing:** Multiple processors execute tasks at the same time, rather than a single processor switching between them.
- **Shared or distributed memory:** Processors can either share a common RAM in a shared memory multiprocessor or have their own private memory in a distributed memory multiprocessor.

Multiprocessors

- **Increased performance:** Workloads can be divided among multiple CPUs, significantly speeding up execution.
- **Enhanced reliability:** If one processor fails, the others can continue to operate, preventing the system from shutting down completely.
- **Scalability:** Additional processors can be added to the system to handle increased workloads.

- **Types of multiprocessors**
- **Shared-memory multiprocessor:** All CPUs have access to a common RAM.
- **Distributed-memory multiprocessor:** Each CPU has its own private memory, and processors communicate through a network.

Multi-Core system

- A processor that has more than one core is called Multicore Processor. In this article we will learn about multicore processor in detail.
- A multicore processor is an integrated circuit for faster processing of several tasks like decreased power consumption and increased performance. it has two or more processors that read and perform program instructions.
- In other words, a multicore processor comprises multiple processing units, or "Cores," each of which has the potential to do distinct tasks.
- For example, if you are doing many tasks at a time such as watching a movie and using other applications. one core processor will handle activities like watching a movie while the multicore processor handles other responsibilities at the same time.

Multi-Core system

- **Uses of Multicore Processor**
- Multicore processors are used in many devices like desktop, laptop, smartphone, and gaming systems. Some applications which use multicore processor are as below.
- Multicore Processor is used in high graphics games like Overwatch and Star Wars Battlefront, and other 3D games.
- The multicore processor is more appropriate used in video editing software like Adobe Photoshop, and iMovie.
- Multicore Processor is used solidworks with computer-aided design (CAD).
- Database servers also handled by multicore CPU.
- Multicore CPU is used in high network traffic.
- Embedded systems can handle by multicore processor.

Multi-Core system

- Advantages of Multicore Processor
- Performance: A multi-core CPU can perform more work as compared to a one-core processor. so multi core processor performance is better.
- Reliability: The software is always assigned to different cores in multi core processor. If one piece of software fails others remain unaffected.
- Software Interactions: If a software is running on many cores. it will communicate with each other.
- Multitasking: Multi core CPU can perform multiple tasks at a same time even if many applications may be run at the same time.
- Power Consumption: A multi-core processor consumes less power. Only a part of the CPU that produces heat will be used. Due to low battery utilization the power consumption is automatically reduced.

Multi-Core system

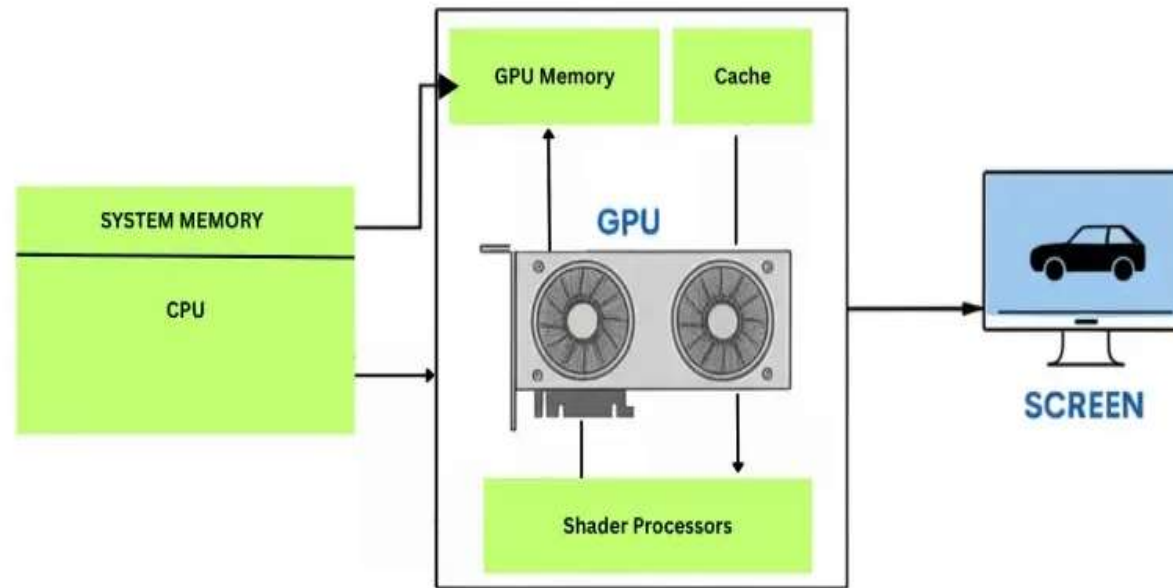
- Disadvantages of Multicore Processors
- Application Speed: A multi-core processor is designed for multitasking, its performance is not enough. when an software is processing It jump from one core to the next. so the result is the cache fills up, increasing its speed.
- Jitter: In a multi-core CPU, the cores increase more interference happens due to this result in excessive jitters. operating system's program performance may suffer, and failures frequency may increase .
- Analysis: When you are doing multiple task at once, you will need to add memory. In a multi-core CPU, this is tough to analysis it.
- Resource Sharing: A multi-core CPU shares multiple resources, both internal and external. Like networks, system buses, and main memory are some resources. Any software running on the same core again and again it will interrupted.

Introduction to Graphics Processing Unit

- Graphics processing technology has evolved significantly over the years, making computers much better at handling images and complex tasks.
- The newest Graphics Processing Units (GPUs) have opened up new opportunities in many areas, like gaming, machine learning, and content creation.
- Today, GPUs are important not just for entertainment but also for research, professional design, and data-driven projects.
- **How Do GPUs Work?**
- Main Function: A GPU (graphics processing unit) accelerates rendering of images, videos, and animations by handling many calculations in parallel. It's also used in machine learning and scientific computing for the same reason.
- Massive Parallelism: GPUs have thousands of small cores (like CUDA cores for NVIDIA, stream processors for AMD) that can process many data streams simultaneously, ideal for parallel workloads.
- **Core Components:**
- GPU Die: Central processing unit of the graphics card that houses all compute units.
- VRAM: Dedicated high-speed memory that stores graphics data and textures for fast access.
- Cooling System: Keeps the GPU from overheating during operation.
- Power Regulation & Interfaces: Provide power and connect the GPU to the motherboard and

Introduction to Graphics Processing Unit

- Graphics processing technology has evolved significantly over the years, making computers much better at handling images and complex tasks.



Introduction to Graphics Processing Unit

➤ Features of GPU

- These features show how GPUs improve visuals, performance, and help with advanced tasks in many different fields.
- **Rendering 2D and 3D Graphics:** GPUs are designed to generate high-quality, smooth visuals for games, movies, and interactive applications, ensuring a realistic experience.
- **Supporting Advanced Software:** High-performance software, such as AutoCAD and professional video editing tools, rely on GPUs for fast processing and seamless performance.
- **Cross-Device Functionality:** GPUs are used in smartphones, tablets, gaming consoles, and TVs, providing crisp, vibrant, and high-definition displays across different devices.
- **Handling Complex Color Processing:** GPUs support advanced color formats like YUV, enhancing the vibrancy and accuracy of images and videos.
- **Accelerating AI and Machine Learning:** GPUs are increasingly used to speed up AI and machine learning tasks, enabling faster processing of large datasets through parallel computing.
- **Uses of GPU**
- GPUs are typically utilized to power the top-notch gaming experiences by producing required incredibly smooth and lifelike graphics and rendering to the user. Nevertheless, a large number of corporate applications or configurations also require powerful graphics processors.

Introduction to Graphics Processing Unit

- **The important uses are mentioned below:**
- **Machine Learning & AI:** GPUs accelerate AI tasks like image recognition and data processing by handling many tasks at once, speeding up model training and improving performance.
- **Gaming:** In gaming, GPUs create realistic, smooth environments with high-resolution graphics, making games more immersive, especially with advancements like 4K displays and virtual reality.
- **Content Creation & Video Editing:** For graphic designers and video editors, GPUs speed up rendering and applying effects, allowing faster, high-quality content creation without lag.
- **Data Science & Big Data:** GPUs process large datasets quickly, helping data scientists analyze information faster and run complex algorithms more efficiently.
- **Cryptocurrency Mining:** GPUs are crucial in cryptocurrency mining, solving the complex calculations needed to secure transactions and earn rewards.
- **Scientific Research & Simulations:** GPUs speed up scientific simulations in fields like physics and biology, enabling faster discoveries and more accurate models.
- **Virtual Reality (VR) & Augmented Reality (AR):** In VR and AR, GPUs deliver smooth, interactive experiences by rendering high-quality visuals in real time.
- **Medical Imaging:** GPUs help process and analyze medical images (like MRIs) quickly, aiding faster, more accurate diagnoses.